

COMPUTER SCIENCE TRIPOS Part IB – 2024 – Paper 4

5 Programming in C and C++ (djg11)

- C/C++ (a) Are C functions exactly the same as C++ functions? What is the difference between a C function and C++ method? [3 marks]

Answer: Yes, for the common language parts, top-level functions are compiled the same in general, except that the generated assembly squirrels the type signature into the label. For instance `int foo(int p)` is called `_Z3fooi` under g++ but is just called `foo` from gcc. The type checking of void arg signatures differs. Also C++ has pass-by-reference and many other assets from C++ , such as templating, default values and exceptions. The difference with a C++ method is a hidden extra argument, the `this` pointer, which gives access to the fields of the class.

- Implementation specific variation (b) The C language is used on a range of microcontrollers where the number of bits used for the `char` type might have any value between 8 and 16, but `int` always uses 24 bits. Write a program that determines the number of bits present in a `char`. [3 marks]

Answer: The following program printed 256 on my laptop. Log to the base 2 of this number is the requested answer.

```
int count = 0; char c = 127;
do { c+= 1; count += 1; } while (c != 127);
printf ("count is %i\n", count);
```

This answer behaves the same with signed or unsigned chars, although strictly, as Tobias and some sanitisers point out, it relies on overflow wrapping which is an undefined behaviour that does not work on all hardware or compilers. Similar points apply to other techniques based on detecting non-linear counting patters, left shifting or fixed point detection. The correct approach in the real-world is to check `limits.h` or `std::numeric_limits<char>::min()`. Directly using these is arguably not determining the answer via a program-writing exercise. Approaches that only read char variables without assignment will not work: unlike Verilog expression width rules, C promotes chars to integers for all arithmetic. [NB. `sizeof` will typically round the answer to a multiple of 8 and is insufficient.]

- (c) On another computer, `char` variables are 8-bits in size, but it is not known if they are signed and unsigned. Write a program to test which. [3 marks]

Answer:

```
int signedf = 0;
char c = 127;
do { c+= 1; if (c<0) signedf = 1; } while (c != 127);
printf("char is %s\n", (signedf)? "signed": "unsigned");
```

Answers based on casting a negative constant to the under-test `char` and checking whether the result remains negative are also acceptable. Another approach uses the 'unsigned wins' arithmetic rule when arguments differ in signedness.

- Objects (d) C++ is object oriented.

- (i) Can C++ upcasts and downcasts always be checked for type safety at compile time? [3 marks]

Answer: In the C and C++ languages, expressions can be freely cast to any types, overriding any type checking. Putting that aside, modern variants are strongly typed and can detect nearly all type errors at compile time, including all upcasts. For downcasts, casting down to a type that does not inherit from the current type will be flagged, but run-time input means many useful downcasts must be tested at run time while others can be statically checked using dataflow analysis.

- (ii) Give an example of a C++ upcast that requires the object pointer to be adjusted. [3 marks]

Answer: Owing to multiple inheritance, where an upcast is not the first-listed parent, we should expect a run-time pointer adjustment. For instance, for the following example on my laptop, the upcast checks whether the pointer is zero and if not takes 8 off of it:

```
struct mother { int m1, m2; }; struct father { int f; };
class child: public mother, public father { int c; };
child *cp = new child;
father *fp = cp; // The upcast
```

Virtual inheritance (anti-diamond) examples also suffice, but this is not just having a virtual method somewhere, which is not sufficient.

- (iii) Give an example where a C++ downcast is checked for validity at run time. What is needed for validity to be checked by the language runtime system? [3 marks]

Answer:

```
mother *cp = new child;
child *fp = dynamic_cast<child *>(cp);
```

This dynamic cast illustrates a basic approach, but does not compile with the above class definitions, saying it cannot ‘dynamic_cast’ since (source type is not polymorphic) so we have to add something like `virtual int foo();` to mother. The generated code (gcc -O0) then invokes `__dynamic_cast` in the C++ run time system to check whether the mother pointer was a child, although in this case it would have been obvious with optimisation turned on. We can alternatively rely on a `bad_cast` exception when using a reference in place of a pointer. A `static_cast` or plain old (C-style) cast *does not* invoke a run-time check and cannot receive full credit.

- Machine model (e) Say why and whether you would expect the following fragment to work:

```
char *mys = "ab_de"; mys[2] = 'c';
```

[2 marks]

Answer: C++ will typically not compile this, requiring a `const char *mys` type definition instead. C may compile it. It still may not work since the string literal may be placed in the program text or constant data segment which is read-only. On embedded systems it will be read-only since it is in ROM and on linux and similar it is read-only for ease of paging when multiple processes run the same binary.
